

THE UNLOX ACADEMY

Week 2 - Pandas + EDA

Indian Startup Funding - Decoded

*40 questions · 100 marks***INDUSTRY CONNECT****What this assessment tests.**

Two weeks ago you knew Python fundamentals. Today you should be able to take a messy real-world CSV - bad column names, mixed-case categories, currency strings, multi-format dates - and produce business insights from it.

Every question below mirrors what an Indian DA/DS analyst does in their first 90 days on the job. The dataset is built around real Indian startups (Flipkart, Zomato, Paytm, BYJU'S, Razorpay, and 80+ others) with deliberately messy formatting injected by us.

Dataset: indian_startups_funding.csv**Format:** Build a .ipynb. Submit as Colab**Time:** Till next assessment

Instructions

1. Total questions: 40. Attempt all. Sections A and B work without the dataset; Section C uses `indian_startups_funding.csv`.
2. Section A is theory - answer in your own words, 2 to 4 lines each. No copy-paste from the internet.
3. Section B is output prediction - write your guess FIRST as a comment, then run the code. If wrong, write a 1-line explanation. Both are graded.
4. Section C is applied coding - use only the techniques covered in Week 2. Do the cleaning ONCE at the top and reuse the cleaned df throughout.
5. Allowed: pandas, numpy. Forbidden: regex, scikit-learn, any chat-analysis or ML library, copy-paste from ChatGPT for any cell. AI-assisted cells must be commented as such.
6. Submission: one .ipynb file uploaded to Google Colab or GitHub. Submit the link via the Google Form Girish shares.
7. Plagiarism check is on. Two matching submissions = both get zero.

Section A – Interview Theory

Write 2 to 4 lines per answer in your notebook as markdown. Be precise. An interviewer can tell within one sentence whether you understand a concept or are paraphrasing a YouTube video.

- Q1.** What's the difference between a Series and a DataFrame? Give one operation that works on both, and one that works only on a DataFrame.
- Q2.** What does `inplace=True` do? Why is the pandas team discouraging its use in newer versions? Give one example where you'd avoid it.
- Q3.** Explain the difference between `.fillna()` and `.dropna()`. When would a data analyst pick one over the other? Be specific - give a concrete situation for each.
- Q4.** What does `pd.to_numeric(col, errors='coerce')` do that plain `pd.to_numeric(col)` does NOT? Which one would you use on real-world messy data, and why?
- Q5.** Explain `.loc` vs `.iloc` with one example each. Then describe a case where `df.loc[0]` and `df.iloc[0]` return different rows.
- Q6.** What is the difference between `.map()`, `.apply()`, and `.replace()` for a Series? When would you reach for each?
- Q7.** What is the `SettingWithCopyWarning`? Give one example of code that triggers it and one way to fix it. (Hint: the chained-assignment pattern.)
- Q8.** Explain `pd.cut()` vs `pd.qcut()`. If you have annual income data and want to label customers as Low / Mid / High earners, which one fits better? Why?

Section B – Predict the Output

For each block: write your predicted output **FIRST** in a comment, then run it. If your prediction was wrong, write a one-line explanation. Both prediction AND explanation are graded.

Q9. Predict the output. What's in df at the end?

```
import pandas as pd
df = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6]})
df2 = df
df2['a'] = [10, 20, 30]
print(df)
```

Q10. Predict both outputs.

```
import pandas as pd
import numpy as np
s = pd.Series([1, 2, np.nan, 4, 5])
print(s.mean())
print(s.sum())
```

Q11. Predict the dtype and the sum.

```
import pandas as pd
df = pd.DataFrame({'price': ['100', '200', 'NA', '400']})
df['price'] = pd.to_numeric(df['price'], errors='coerce')
print(df['price'].dtype)
print(df['price'].sum())
```

Q12. Predict the type of df.loc[0].

```
import pandas as pd
df = pd.DataFrame({'name': ['Rahul', 'Priya'], 'age': [22, 21]})
print(df.loc[0])
print(type(df.loc[0]))
```

Q13. Predict – and explain what's wrong with the modification line.

```
import pandas as pd
df = pd.DataFrame({'rating': [4.5, 3.8, 4.2, 5.0]})
df_filtered = df[df['rating'] > 4]
df_filtered['rating'] = df_filtered['rating'] * 2    # what warning?
print(df)
print(df_filtered)
```

Q14. Predict the result AND its dtype. Why is the dtype what it is?

```
import pandas as pd
```

```
df = pd.DataFrame({'a': [1, 2, 3, 4]})
result = df['a'].apply(lambda x: x**2 if x > 2 else 'small')
print(result)
print(result.dtype)
```

Q15. Predict each output. What's the difference between `nunique()`, `len(unique())`, and `value_counts()`?

```
import pandas as pd
df = pd.DataFrame({'city': ['Delhi', 'Mumbai', 'Delhi', 'Bengaluru', None]})
print(df['city'].nunique())
print(len(df['city'].unique()))
print(df['city'].value_counts())
```

Q16. Predict each line. The commented line will throw an error or silently break — explain why.

```
import pandas as pd
df = pd.DataFrame({'a': [1, 2, 3], 'b': [10, 20, 30]})
print(df[df['a'] > 1])
# print(df[df['a'] > 1 & df['b'] > 15]) # <-- what's wrong here?
print(df[(df['a'] > 1) & (df['b'] > 15)])
```

Section C – Applied on Indian Startups Dataset

All questions use `indian_startups_funding.csv`. The file contains 468 rows of Indian startup funding records (real Indian companies plus made-up ones), with intentionally messy column names, city aliases, industry variants, funding amounts as strings, and dates in 4 different formats. Your first job is to clean it.

INDUSTRY CONNECT

The cleaning expected of you (do this ONCE at the top of Section C).

1. Rename columns to clean snake_case: `startup_name`, `city`, `industry`, `funding_stage`, `amount`, `funding_date`, `investors`, `founded_year`.
 2. Clean `startup_name` and `city` - strip whitespace, title case.
 3. Standardise city aliases: Bangalore / BLR → Bengaluru; Bombay → Mumbai; Gurgaon → Gurugram; Delhi → New Delhi; Calcutta → Kolkata; Madras → Chennai.
 4. Standardise industry and `funding_stage` variants using dicts.
 5. Convert amount to numeric (in INR Cr): strip ₹, INR, Cr, Crore, commas, then `pd.to_numeric(errors='coerce')`. 'Undisclosed' should become NaN.
 6. Convert `funding_date` to datetime with `errors='coerce'` and `dayfirst=True`.
 7. Drop duplicate rows.
- Use this cleaned df for every question that follows. Anyone who skips cleaning cannot answer Q22 onwards correctly.

Loading & Inspection (Q17 to Q19)

Q17. After loading the raw CSV (before cleaning), answer:

- (a) How many rows and columns?
- (b) How many missing values are in each column?
- (c) How many exact duplicate rows are present?

Q18. After cleaning column names to snake_case, print the cleaned column names sorted alphabetically. Don't manually type them – use Python on `df.columns`.

Q19. Without using `.describe()`, compute these statistics on the cleaned `founded_year` column (after dropping NaN): count, mean, min, max, median, std. Print as a single dictionary with these exact keys.

Cleaning Verification (Q20 to Q24)

Q20. After cleaning the city column with the aliases mapped, print all unique city names sorted alphabetically. There should be exactly 10 cities and no duplicates from case or whitespace.

Q21. After standardising the industry column using a dict, print the count of startups per industry sorted from most to least common (use `value_counts()`).

Q22. After converting amount to numeric, print:

- (a) The dtype of the column
- (b) How many NaN values are there (from 'Undisclosed' entries)
- (c) The mean funding amount (ignoring NaN)
- (d) The minimum and maximum funding amount

Q23. After converting funding_date to datetime:

- (a) Count how many deals happened in each year. Use `.dt.year`.
- (b) Print the year with the highest number of funding deals.

Q24. After standardising funding_stage, print the count of startups per stage (sorted from most common to least).

Filtering (Q25 to Q28)

Q25. Find all Bengaluru-based startups with funding amount > 100 Cr. Print the startup_name and amount columns, sorted by amount descending.

Q26. Using `.isin()`, find startups whose industry is one of: Fintech, Edtech, HealthTech. Print the total count.

Q27. Using `.between()`, find startups founded between 2015 and 2020 (both inclusive). Print the count. Be careful – founded_year has missing values.

Q28. Using `.str.contains()` with `case=False` and `na=False`, find all startups whose name contains the word 'pay'. Print the unique names.

Transformation & Derived Columns (Q29 to Q32)

Q29. Add a new column `funding_efficiency = amount / (2024 - founded_year)`. Higher = raised more capital per year of existence. Sort by `funding_efficiency` descending. Print top 10 with name, industry, amount, founded_year, and funding_efficiency.

Q30. Using `.apply()` with a lambda, add a `funding_tier` column based on amount:

- amount ≥ 1000 Cr → 'Unicorn'
- amount ≥ 500 Cr → 'Mega'
- amount ≥ 100 Cr → 'Large'
- amount ≥ 10 Cr → 'Mid'
- else (including NaN) → 'Small / Unknown'

Print the count of startups in each tier.

Q31. Using `pd.cut()`, bin founded_year into 3 eras:

- Before 2010 → 'Early'
- 2010 to 2017 → 'Growth'

2018 and later → 'Recent'

Print the count of startups in each era.

Q32. The investors column has multiple investors separated by ', ' (comma + space). Create a new column `investor_count` showing the number of investors per startup. Use `.apply()` with a lambda. Handle NaN values (count = 0). Print the `value_counts()` of `investor_count`.

Aggregation & Grouping (Q33 to Q37)

Q33. Compute the average funding amount per industry (ignoring NaN). Sort descending. Print the top 5 industries by average funding.

Q34. For each city, compute three metrics in a single `groupby` using `.agg()` with a dictionary:

- (a) Number of startups
- (b) Total funding raised (sum of amount)
- (c) Average funding per startup

Print the top 10 cities by number of startups.

Q35. Interview-style. Which industry has the highest average funding, but only considering industries with at least 15 startups? (Sparse industries bias the mean.) Print industry name, count, and average funding.

Q36. Using `pd.crosstab()`, show the distribution of `funding_tier` across the top 5 industries by count. Rows = industry, columns = `funding_tier`, values = count of startups.

Q37. For each industry, find the startup with the highest funding amount. Print `startup_name`, industry, city, and amount. Hint: `groupby + idxmax()` on amount.

Real Industry Problems (Q38 to Q40)

Q38. Emerging player finder. An 'emerging player' is a startup founded in 2019 or later, with funding ≥ 100 Cr, in one of Fintech / Edtech / HealthTech. Find all of them, sort by amount descending, print name, industry, city, `founded_year`, and amount. Why might a VC analyst care about this list?

Q39. Investor hub analysis. A foreign investor wants to know which Indian city has the highest TOTAL funding raised, considering only cities with at least 20 startups (so the number is meaningful). Print city name, count, and total funding.

Q40. Market saturation. For each combination of (city, industry), count the number of startups. Print the top 10 most-saturated combinations. This tells a new founder where competition is most intense – and by elimination, where the gaps might be.

Submission Guidelines

Format your submission as follows so it can be graded quickly:

1. One file: Week2_Assessment_<YourName>.ipynb
2. Each question separated by a comment header: # ===== Q1 =====
3. Sections A theory answers – as markdown cells.
4. Section B – prediction as a comment FIRST, then code, then 1-line explanation if wrong.
5. Section C – clean the dataset ONCE at the top, then answer each question using the cleaned df.
6. Upload to Google Colab or GitHub. Make sure the share setting is 'Anyone with link can view'.
7. Submit ONLY the LINK via the Google Form in the LMS portal.

INDUSTRY CONNECT

Grading rubric - total 100 marks.

Section A – Theory: 24 marks (3 per question). Marks for accuracy and precision of explanation, not for word count.

Section B – Output Prediction: 24 marks (3 per question). 2 for correct prediction, 1 for correct explanation. Wrong prediction with right explanation still gets 1.

Section C – Applied: 48 marks (2 per question). Code that produces the wrong answer gets zero on that question, even if the approach was almost right.

Code quality: 4 marks. Variable names, comments where logic is non-obvious, clean cell organisation.

Pass: 65. Distinction: 85+.

Constraint violation (using regex, scikit-learn, or any forbidden library) = automatic 0 on that question, even if the answer is correct.

All the best. Be honest with yourself – If you needed AI help on a question, note it in the comment. We track growth, not perfection.